# Benchmarking de Algoritmos de Ordenação: Da Teoria Assintótica à Prática

Uma análise empírica de performance sob diferentes condições de entropia.

>_ Author: Lucas Nascimento Aguiar | RA: 10766622

NotebookLM

# Parâmetros de Telemetria e Setup Experimental

## Stack & Hardware

- Runtime: Go 1.26.1
- OS: Linux (WSL2)
- CPU: AMD Ryzen 5 3600X (6c/12t)

## Alvos (Algoritmos)

**O(n^2):** Bubble, Selection, Insertion

**O(n log n):** Merge, Quick

## Carga (Volume de Dados)

10^3 elementos únicos

10^4 elementos únicos

10^5 elementos únicos

## Entropia & Execução

Cenários: Crescente, Decrescente, Aleatório (Fisher-Yates)

Rigor: **5 execuções**, medição em **nanossegundos**

NotebookLM

# A Grande Divisão: O Peso da Escala em 10^5 Elementos

Mapa de calor dos tempos médios (ms)

| Algorithm | random_shuffled·1000 | random_shuffled·10000 | random_shuffled·100000 | sorted_ascending·1000 | sorted_ascending·10000 | sorted_ascending·100000 | sorted_descending·1000 | sorted_descending·10000 | sorted_descending·100000 |
|---|---|---|---|---|---|---|---|---|---|
| bubble | 0.5 | 57.2 | 12126.8 | 0.0 | 0.0 | 0.1 | 0.8 | 82.0 | 7774.6 |
| insertion | 0.1 | 11.3 | 1093.3 | 0.0 | 0.0 | 0.2 | 0.3 | 27.8 | 2345.4 |
| merge | 0.1 | 0.9 | 11.2 | 0.1 | 0.9 | 7.2 | 0.0 | 0.5 | 7.0 |
| quick | 0.0 | 0.5 | 6.3 | 0.5 | 35.3 | 3722.8 | 0.4 | 38.5 | 3545.3 |
| selection | 0.4 | 37.1 | 3456.0 | 0.4 | 41.1 | 3500.0 | 0.4 | 40.8 | 3369.7 |

Sample-Size

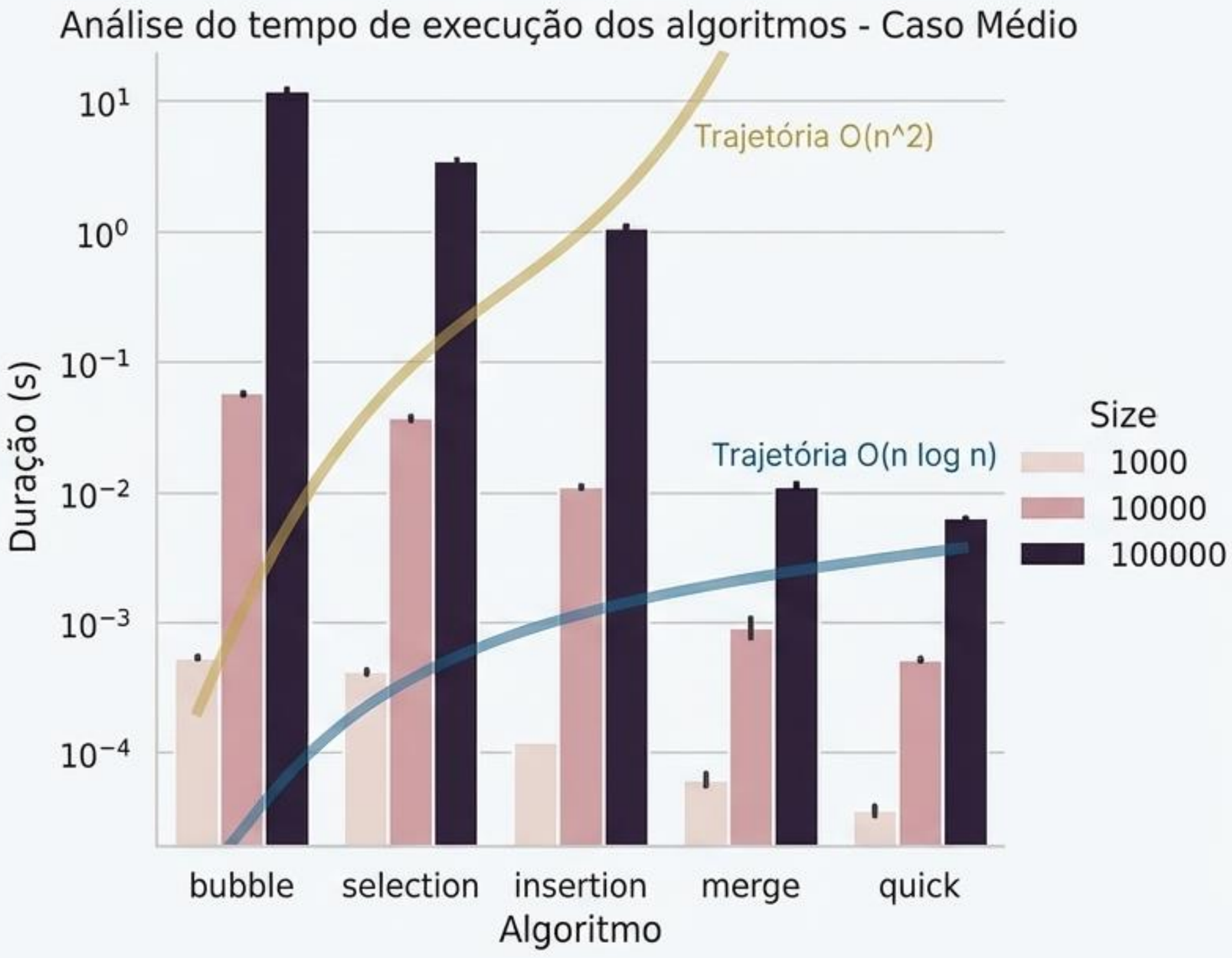
Pior tempo:
>12s de execução

Escalabilidade
O(n log n)
preservada

A prática confirma a teoria assintótica: algoritmos O(n^2) tornam-se inviáveis para volumes industriais, independentemente da eficiência da linguagem Go.

A) NotebookLM

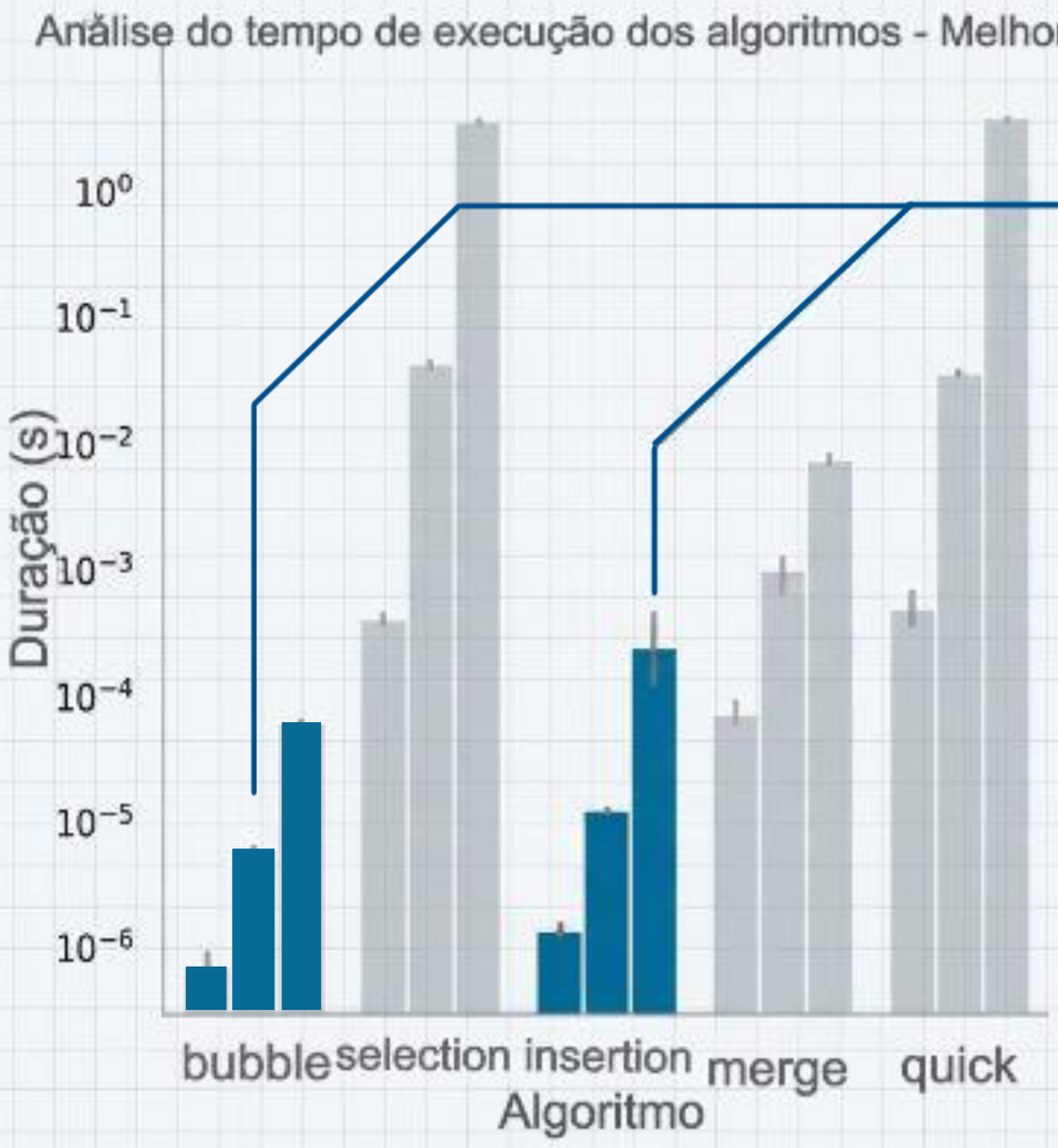
# O Cenário Padrão: Comportamento sob Entropia Aleatória




Em condições normais (dados embaralhados aleatoriamente), a diferença de ordem de grandeza prova que a eficiência estrutural supera qualquer otimização de baixo nível da linguagem. Bubble e Selection sofrem colapso prematuro.

NotebookLM

# O Paradoxo do Melhor Caso: A Ressurreição do Insertion Sort



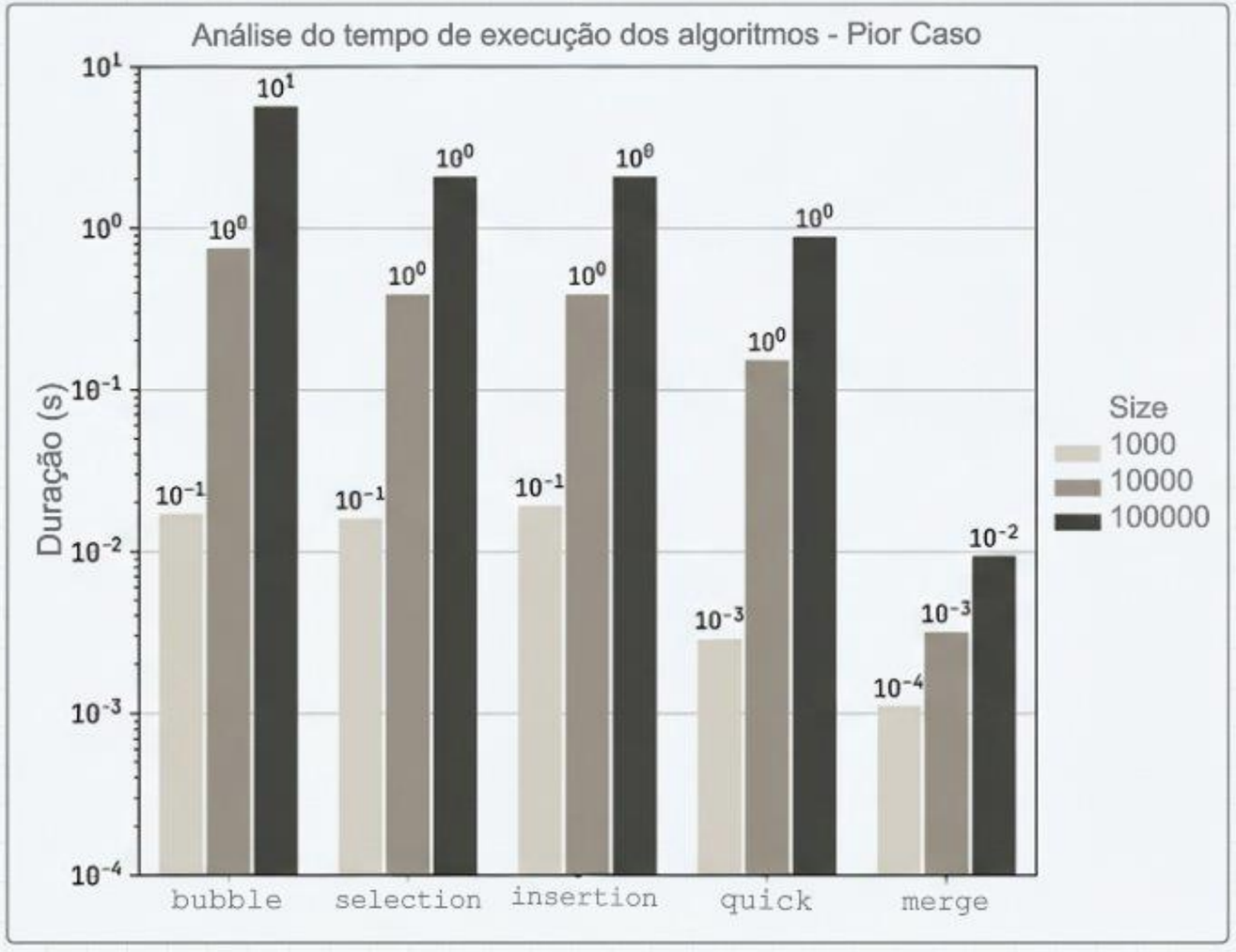

**Comportamento:**
**Próximo ao linear ~ 0(n)**

## Diagnóstico:

Quando a entropia é zero (dados ordenados), o branch prediction do processador e o early exit do laço interno do Insertion/Merge Sort superam o overhead de recursão do Quick e Merge Sort.

A NotebookLM

# O Teste de Estresse: Degradação e Vulnerabilidade no Pior Caso




## Degradação do Quick Sort

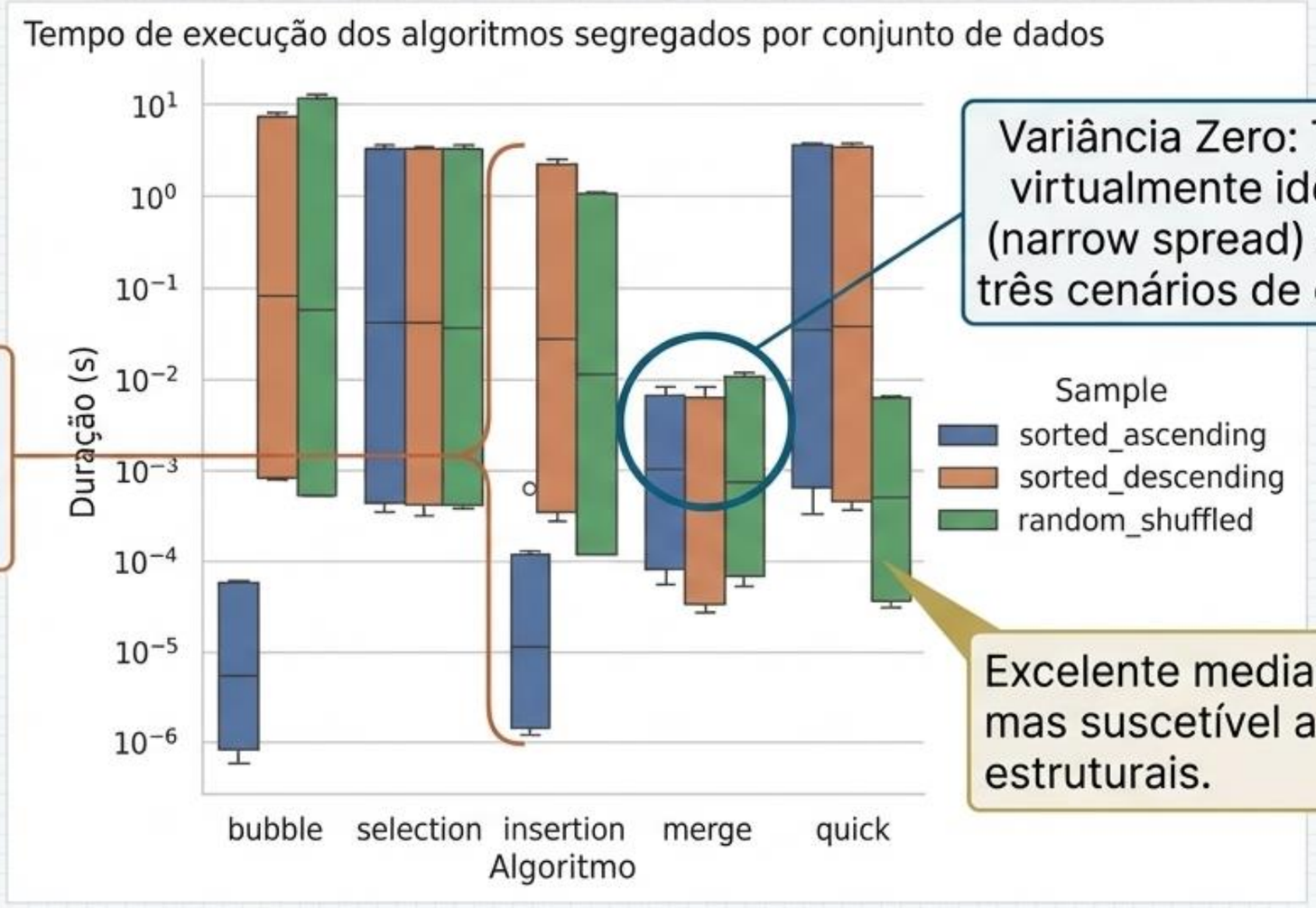
Ao receber dados inversamente ordenados, a escolha do pivô (n-1), aproxima a execução do comportamento quadrático.

## Previsibilidade no Merge Sort

Mantém desempenho estável e previsível de O(n log n), confirmando sua absoluta indiferença à configuração inicial do vetor.

A NotebookLM

# Assinaturas de Variância: A Sensibilidade à Entropia




Variância Zero: Tempos virtualmente idênticos (narrow spread) entre os três cenários de entropia.

Variância Extrema: De **O(n)** a **O(n^2)** dependendo apenas do input.

Excelente mediana geral, mas suscetível a outliers estruturais.

**Conclusão:** Algoritmos O(n log n) garantem um teto de processamento, mas a variância interna do desempenho ainda é ditada pela estrutura da entrada.

NotebookLM

# **Matriz de Diagnóstico Tático** (Framework de Decisão)

| Algoritmo | Uso Ideal | Risco Empírico |
|---|---|---|
| Merge Sort | Qualquer volume de dados onde a previsibilidade absoluta (tempo garantido) é mais importante que economizar memória. | ✔Nenhum sob estresse de CPU (estável em todos os cenários). |
| Quick Sort | Grandes volumes de dados predominantemente aleatórios. | ⚠️ Colapso de performance se a escolha do pivô coincidir com dados já ordenados ou invertidos. Existem estratégias de balanceamento de pivô que otimizam o algoritmo. |
| Insertion Sort | Micro-lotes de dados ou rotinas contínuas em vetores quase totalmente ordenados. | ⚠️ Letal em dados de grande volume ou ordem inversa. |
| Bubble / Selection | - | ❌ Desempenho consistentemente inferior; inviáveis para produção real. |

NotebookLM

Códigos fonte, dados e análises, podem ser encontrados no Github

```
$ git clone https://github.com/olucasaguiar/projeto-metodos-de-
ordenacao.git
$ . /run.sh # go run "app/benchmark.go" {algoritmo}
```

Obrigado!

NotebookLM